EMBLEM-NLP RESEARCH NOTE

The Tokenomics of Data-Serialization Formats

A cross-provider engineering reference to serialization-format efficiency, tokenizer behavior, pricing, and reliability.

June 2026

SECTION 01

Executive Summary

This report compares serialization formats across two public benchmarks and a controlled provider-pricing benchmark to understand their impact on tokens, cost, and accuracy.

63%

CSV used 62.9% fewer tokens than pretty JSON in the TOON 100-employee benchmark.

2.7×

Markdown-KV consumed 2.7× CSV's tokens but led the ImprovingAgents accuracy benchmark.

5–6×

Output tokens cost 5–6× input on current OpenAI, Gemini Pro, and Claude representative models.

12

Figure 2 plots TOON and every other format from one internally-consistent benchmark setup.

BOTTOM LINE

1. **Flat, high-volume input:** CSV or TSV is the cost floor; TOON is a structured alternative when uniform arrays justify a schema-once representation.
2. **Nested or heterogeneous input:** compact JSON is the safest efficiency baseline. TOON can become larger than compact JSON on mixed or deeply nested structures.
3. **Structured output:** keep JSON Schema or strict tool calling when validation matters. OpenAI, Claude, Gemini, and xAI provide schema-constrained output mechanisms.
4. **Benchmark discipline:** never combine token counts and accuracy from different datasets, tokenizer families, or serialization settings in the same scatter plot.

Publication correction. The four previously-missing ImprovingAgents token counts were recovered from the original article and TOON was added from the same GPT-4.1-nano follow-up, producing a fully-sourced 12-format table. The accuracy data is attributed to ImprovingAgents (not an EMBLEM-NLP benchmark), and the run is one model — GPT-4.1-nano — not six providers.

Contents & Revision Scope

Δ **What changed in Rev 2.1.** Figure 2 rebuilt from the complete 12-format table including the later TOON run; provider pricing refreshed to official June 16, 2026 rates (xAI flagship is Grok 4.3 at $1.25/$2.50; Mistral Large 3 at $0.50/$1.50; DeepSeek V4 Pro corrected to $0.435/$0.87); same-record counts upgraded from inferred to measured; cost-at-scale uses the benchmark's sourced 37.1% retained ratio.

Method and Comparability

Our analysis is grounded in three complementary evidence classes. Each is defined below to support consistent interpretation and cross-provider comparability.

01 **TOON token-efficiency benchmark:** 100 uniform employee records, tokenized with the GPT-5 o200k_base tokenizer. Compares CSV, TOON, compact JSON, YAML, pretty JSON, and XML.

02 **ImprovingAgents comprehension benchmark:** 1,000 synthetic employee records, eight attributes per record, 1,000 randomized field-retrieval questions, GPT-4.1-nano, and 12 paired format results when the TOON follow-up is included (11 in the original table).

03 **Provider pricing:** current list prices from each provider's official pricing documentation, verified June 16, 2026, quoted in USD per million tokens.

§ **Rule:** use a single benchmark's paired token and accuracy values for any accuracy-versus-cost visualization. Token counts from the TOON benchmark are not substituted into the ImprovingAgents scatter.

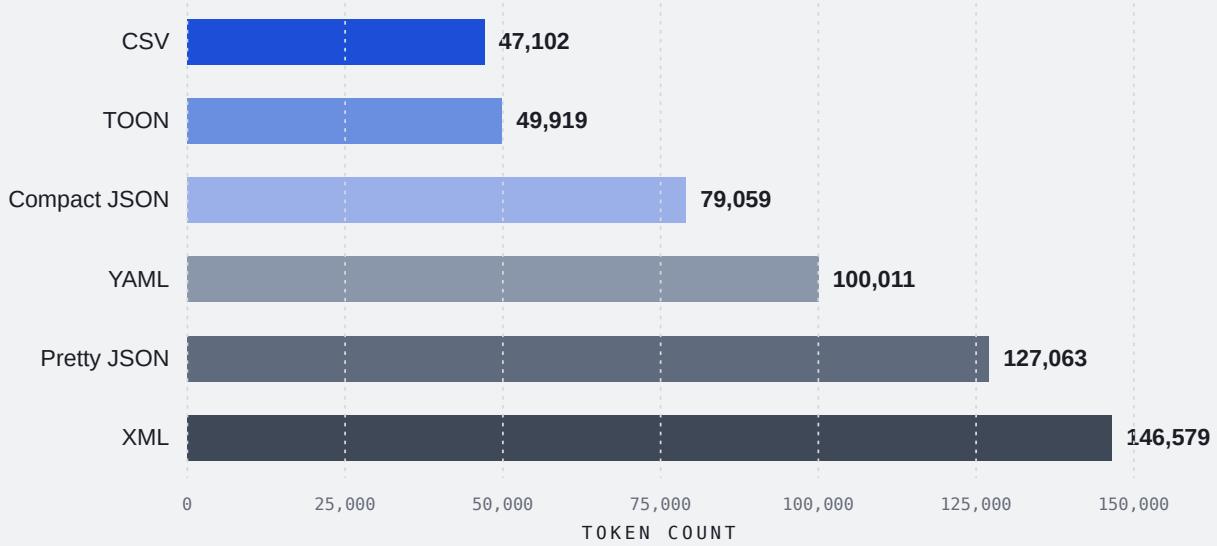
TOKENIZER PROTOCOL

Absolute counts are tokenizer-dependent. The TOON benchmark explicitly uses o200k_base. ImprovingAgents did not disclose its tokenizer; o200k_base is a strong model-matched inference for GPT-4.1-nano, not a confirmed methodology fact. For production measurement, use the target provider's tokenizer or count endpoint and preserve the exact serializer configuration in the benchmark record. `[evidence: M·D·V labeled per claim in sources.csv]`

SECTION 03

Token Cost by Format

FIGURE 1. TOKEN COST BY FORMAT — 100 EMPLOYEE RECORDS



CSV is the least-expensive representation in this flat-data case. TOON is 6.0% larger than CSV, but 36.9% smaller than compact JSON and 60.7% smaller than pretty JSON. XML is the most expensive because element names appear in both opening and closing tags.

Figure 1. Benchmark-wide aggregate token counts for the TOON flat-track 100-employee workload. Counts use the GPT-5 o200k_base tokenizer. These are not the single serialized-payload counts shown on page 9. Evidence: M (measured). Ledger IDs F1-*.

Serialization Formats and Trade-offs

Different serialization formats impose different overhead patterns that directly affect token position and cost. Choosing the right format depends on data shape, usage context, and downstream requirements.

FORMAT FAMILY	MAIN OVERHEAD	TYPICAL TOKEN POSITION	BEST FIT
CSV / TSV	Header once; delimiters and newlines	Lowest for flat tables	Flat, uniform rows
TOON	Schema-once tabular encoding	Very low on uniform arrays; can lose to compact JSON on mixed/nested data	Large, repeated uniform input
Markdown table / KV	Pipes or repeated key-value labels	Low to medium	Reasoning-heavy prompts and human review
Compact JSON	Repeated keys, no optional whitespace	Medium	Robust default and machine interchange
YAML	Indentation and dash/colon markers	Medium; result depends on JSON baseline	Human-authored config
Pretty JSON	Repeated keys plus indentation	High	Debugging and inspection
XML	Opening and closing tags	Highest in the cited flat benchmark	Legacy integration and document markup

KT

Shape sensitivity: on the same official benchmark suite, TOON is **14.7%** larger than compact JSON across the mixed-structure track. Its advantage is concentrated in uniform, tabular arrays rather than arbitrary JSON-shaped data.

SECTION 05

Accuracy versus Token Cost

We compare 12 serialization formats on accuracy and token cost using one internally-consistent benchmark run — the ImprovingAgents study on a single model, GPT-4.1-nano.

FIGURE 2. ACCURACY VERSUS TOKEN COST — SAME BENCHMARK RUN

Figure 2. All 12 paired formats from the same ImprovingAgents GPT-4.1-nano setup (1,000 records × 1,000 queries). TOON was added in the later follow-up under the identical harness. X-axis logarithmic; dashed lines are medians used only as visual quadrant guides. SOURCE: ImprovingAgents, "Which Table Format Do LLMs Understand Best? (11 Formats)" — improvingagents.com/blog/best-input-data-format-for-llms/. Evidence: M. Ledger IDs F2-*.

Complete 12-Format Benchmark Table

TABLE 1. ACCURACY & TOKEN COST SUMMARY

#	FORMAT	ACCURACY	TOKENS	RANK
1	**Markdown-KV**	60.7%	52,104	1
2	**XML**	56%	76,114	2
3	**INI**	55.7%	48,100	3
4	**YAML**	54.7%	55,395	4
5	**HTML**	53.6%	75,204	5
6	**JSON**	52.3%	66,396	6
7	**Markdown table**	51.9%	25,140	7
8	**Natural language**	49.6%	43,411	8
9	**TOON**	47.5%	21,518	9
10	**JSONL**	45%	54,407	10
11	**CSV**	44.3%	19,524	11
12	**Pipe-delimited**	41.1%	43,098	12

SOURCE: ImprovingAgents, GPT-4.1-nano, 1,000 records × 1,000 queries. Evidence: M (measured). Not an EMBLEM-NLP-original benchmark.

KEY INTERPRETATION

The benchmark demonstrates a real cost-accuracy trade-off.

CSV is the cheapest at 19,524 tokens but scored 44.3%.

Markdown-KV scored 60.7% at 52,104 tokens.

Markdown tables were the strongest compact compromise at 51.9% and 25,140 tokens; **TOON** reached 47.5% at 21,518 tokens, with no statistically-significant accuracy difference from CSV.

Interpretation: this is not a universal format ranking. It shows that, for one small model, one synthetic dataset, and one question type, familiarity and explicit structure affected retrieval accuracy enough to outweigh raw compression.

Same Record, Every Format

The same 8-attribute employee record in six encodings, annotated with **measured** amortized tokens per record (payload total ÷ 100), preserving an apples-to-apples tokenizer and wrapper baseline.

CSV — 23.3 tok/rec · 2,334 tot [M]

```
id,name,age,city,department,salary,years_experience,project_count
1,Diana A0,46,London,Engineering,141015,7,17
```

TOON — 25.0 tok/rec · 2,498 tot [M]

```
employees[100]
{id,name,age,city,department,salary,years_experience,project_count}:
  1,Diana A0,46,London,Engineering,141015,7,17
```

Compact JSON — 39.2 tok/rec · 3,924 tot [M]

```
[{"id":1,"name":"Diana
A0","age":46,"city":"London","department":"Engineering","
salary":141015,"years_experience":7,"project_count":17}]
```

YAML — 49.6 tok/rec · 4,959 tot [M]

```
employees:
  - id: 1
    name: Diana A0
    age: 46
    city: London
    department: Engineering
    salary: 141015
    years_experience: 7
    project_count: 17
```

Pretty JSON — 63.3 tok/rec · 6,331 tot [M]

```
[
  {
    "id": 1,
    "name": "Diana A0",
    "age": 46,
    "city": "London",
    "salary": 141015
  }
]
```

XML — 73.0 tok/rec · 7,296 tot [M]

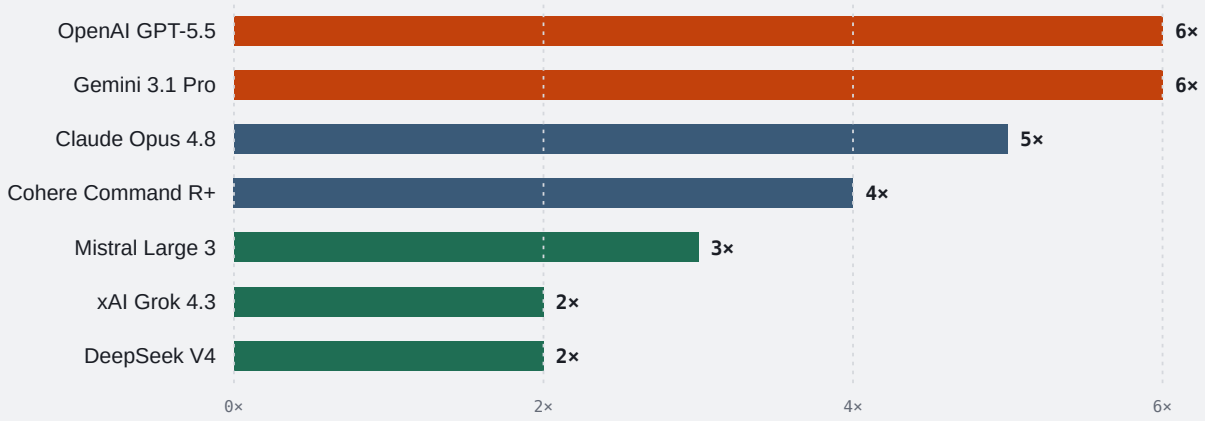
```
<employees>
  <employee id="1">
    <name>Diana A0</name>
    <age>46</age>
    <city>London</city>
    <salary>141015</salary>
  </employee>
</employees>
```

SOURCE: TOON official benchmark, o200k_base tokenizer. Payload-only totals ÷ 100 records: CSV 2,334 · TOON 2,498 · compact JSON 3,924 · YAML 4,959 · pretty JSON 6,331 · XML 7,296. Evidence: M — upgraded from the prior inferred illustration. Distinct from Figure 1's benchmark-wide aggregate. Ledger IDs F5-*.

SECTION 07

Provider Pricing Leverage

FIGURE 3. OUTPUT-TO-INPUT PRICE MULTIPLIERS



Output-format optimization has the largest percentage-dollar leverage on models with high output multipliers. OpenAI GPT-5.5 and Gemini 3.1 Pro price output at 6× input; Claude Opus 4.8 is 5×. Current Grok 4.3 and DeepSeek V4 are 2×, so the same percentage reduction in output tokens saves proportionally less. After xAI retired its "Fast" tier in May 2026, Grok's flat ratio now lives in its flagship.

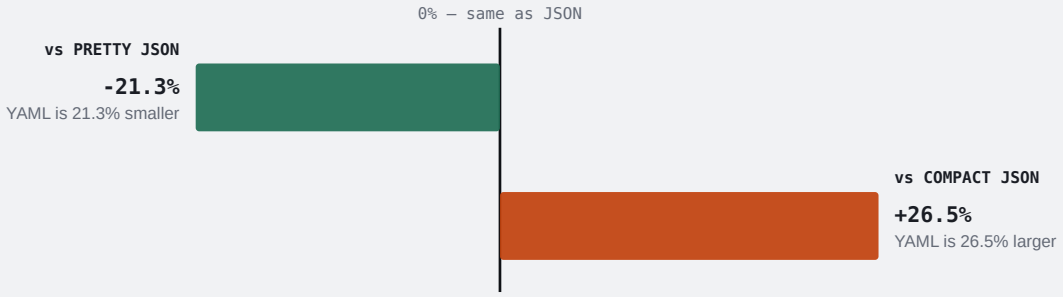
PROVIDER	REPRESENTATIVE MODEL	INPUT $/M	OUTPUT $/M	RATIO
OpenAI	**GPT-5.5**	$5.00	$30.00	6×
OpenAI	**GPT-5.4**	$2.50	$15.00	6×
Anthropic	**Claude Opus 4.8**	$5.00	$25.00	5×
Anthropic	**Claude Sonnet 4.6**	$3.00	$15.00	5×
Google	**Gemini 3.1 Pro (≤200K)**	$2.00	$12.00	6×
Google	**Gemini 3.5 Flash**	$1.50	$9.00	6×
Cohere	**Command R+ (08-2024)**	$2.50	$10.00	4×
Mistral	**Mistral Large 3**	$0.50	$1.50	3×
xAI	**Grok 4.3**	$1.25	$2.50	2×
DeepSeek	**V4 Pro**	$0.435	$0.87	2×
DeepSeek	**V4 Flash**	$0.14	$0.28	2×

Official rates verified June 16, 2026. Evidence: V (vendor-reported) for prices, D (derived) for ratios. Resolved conflicts: Grok 4.3 supersedes Grok 4 ($3/$15); Mistral Large 3 supersedes Large 2 ($2/$6); DeepSeek V4 Pro corrected from $1.74/$3.48. Ledger IDs F3-*.

SECTION 08

The YAML Baseline Flip

FIGURE 4. YAML TOKEN DELTA AGAINST TWO JSON BASELINES



The phrase "YAML is more token-efficient than JSON" is incomplete until the JSON baseline is named. Within one TOON 100-employee dataset, YAML is 21.3% smaller than pretty JSON but 26.5% larger than compact JSON. Optional JSON whitespace can be removed; YAML indentation and line structure are part of its syntax. For automated prompt assembly, compact JSON is the relevant baseline.

✓ **Production rule:** minify JSON before evaluating a format migration. Otherwise the benchmark mostly measures removable whitespace rather than the serialization grammar.

SOURCE: TOON 100-employee benchmark, o200k_base. Exact within-dataset comparison. Evidence: M / D. Ledger IDs F4-*.

SECTION 09

Cost at Scale

An illustrative input-only scenario. CSV's 37.1% retained-token ratio is taken from the TOON 100-employee benchmark; Claude Sonnet 4.6 input is $3/M tokens.

MONTHLY TABULAR INPUT	PRETTY JSON	CSV ESTIMATE	ESTIMATED SAVING
10M tokens	$30.00	$11.12	$18.88
100M tokens	$300.00	$111.21	$188.79
1B tokens	$3,000.00	$1,112.09	$1,887.91

> ! This is not a promise of production savings. It assumes the workload has the same flat-data shape and token ratio as the benchmark, excludes caching and batch discounts, and treats the stated token volume as the pretty-JSON baseline.

SOURCE: Cost model. Sonnet 4.6 input $3.00/M (verified 2026-06-16). CSV retained-token ratio 37.1% from TOON benchmark (2,334 ÷ 6,331). Evidence: D (derived). Ledger IDs F8-*.

SECTION 10

Format Decision Flow

Is the payload flat and tabular?	→ Use CSV or TSV for input; validate escaping and embedded newlines.
Is it a large, uniform array of objects?	→ Benchmark TOON against CSV and compact JSON using the target model.
Is the structure nested, mixed, or API-facing?	→ Use compact JSON as the default interchange format.
Is the model generating machine-consumed output?	→ Use JSON Schema structured output or strict tool calling.
Is human editing the dominant requirement?	→ Use YAML/TOML for maintainability — not on a blanket token claim.
Are savings material at production volume?	→ If not: prefer operational simplicity; optimize data selection before notation.

⚠ **Output caution:** novel formats carry a prompt tax and repair-loop risk. A 2026 TOON-generation benchmark found instructional overhead can erase token savings on short or structurally-complex outputs.

SECTION 11

Reliability and Structured Output

Token reduction is only economically useful if the downstream system can parse and trust the result. Modern provider APIs increasingly support constrained JSON generation.

O **OpenAI:** Structured Outputs constrain responses to developer-supplied JSON schemas; OpenAI reported 100% schema compliance on its cited complex-schema evaluation for GPT-4o-2024-08-06, vs <40% without the newer mechanism. V

C **Claude:** JSON outputs and strict tool use are generally available on current Claude models, including Sonnet 4.6 and Opus 4.8. V

G **Gemini:** structured outputs accept JSON Schema and integrate with Pydantic and Zod in the SDKs. V

x **xAI:** supported Grok models can return responses guaranteed to match supported JSON-schema features. V

RELIABILITY HIERARCHY

OUTPUT METHOD	TOKEN EFFICIENCY	VALIDATION STRENGTH	OPERATIONAL RISK
Strict JSON Schema / tool call	Medium	Highest	Lowest for machine consumption
Plain compact JSON + validator	Medium	High with retries	Moderate
CSV / TSV + parser + adversarial tests	High for flat output	Medium	Delimiter, quote, newline failures
YAML / TOON generated from prompting	Workload-dependent	Low to medium	Prompt tax, syntax repair, unfamiliarity

Revised conclusion: JSON remains the dominant schema-constrained interchange target. CSV, TSV, YAML, and TOON output usually require custom validation, escaping rules, and retry logic.

SECTION 12

Recommendations

STAGE 1 Safe defaults

- Minify JSON sent to models unless human inspection is required in the prompt transcript.
- Use provider-native JSON Schema or strict function/tool calling for machine-consumed outputs.
- Record serializer version, tokenizer/count endpoint, payload hash, model, and price date with every benchmark.

STAGE 2 Optimize high-volume input

- Use CSV/TSV for flat tables after adversarial tests covering delimiters, quotes, tabs, Unicode, and embedded newlines.
- Evaluate TOON only where arrays are large and uniform enough to amortize schema/header cost.
- For mixed or nested structures, benchmark against compact JSON, not pretty JSON.
- Include comprehension accuracy, parse-failure rate, retries, latency, and total billed tokens in the objective function.

STAGE 3 Output optimization

- Optimize output only after the structured-output baseline is stable and measured at production scale.
- Price repair loops explicitly: expected cost = first-pass tokens + failure probability × retry/repair cost.
- Prefer a hybrid pattern when justified: compact/TOON/CSV input, schema-constrained JSON output.

Caveats & References

CAVEATS

- **External validity:** the ImprovingAgents result uses one model, synthetic employee data, and direct field retrieval. It does not establish universal model behavior.
- **TOON scope:** the largest token gains occur on uniform arrays. Mixed and deeply-nested data can favor compact JSON.
- **Tokenizer variance:** absolute counts change by provider/model even when the ranking is stable.
- **Pricing volatility:** rates are dated snapshots. Batch, cache, flex, and regional tiers can dominate notation savings.
- **Generation vs comprehension:** a format cheap to read may be expensive or unreliable to generate.
- **Data selection dominates notation:** filtering fields, deduplicating rows, and retrieving fewer records often saves more than switching syntax.

REFERENCES

1 **TOON Benchmarks — official benchmark documentation**
https://toonformat.dev/guide/benchmarks

2 **ImprovingAgents — table-format study & TOON follow-up**
https://www.improvingagents.com/blog/best-input-data-format-for-llms/

3 **OpenAI API pricing**
https://openai.com/api/pricing/

4 **Anthropic Claude pricing**
https://platform.claude.com/docs/en/about-claude/pricing

5 **Google Gemini API pricing**
https://ai.google.dev/gemini-api/docs/pricing

6 **xAI models & pricing**
https://docs.x.ai/developers/models

7 **DeepSeek pricing**
https://api-docs.deepseek.com/quick_start/pricing

8 **Mistral pricing**
https://mistral.ai/pricing

9 **Cohere pricing**
https://cohere.com/pricing

10 **OpenAI — Introducing Structured Outputs in the API**
https://openai.com/index/introducing-structured-outputs-in-the-api/

11 **Matveev — TOON vs JSON (arXiv:2603.03306)**
https://arxiv.org/abs/2603.03306

Document status: publication-ready corrected edition. Figures are deterministic vector graphics. The accuracy scatter contains 12 fully-sourced paired values and no estimated or fabricated token counts. Full machine-readable evidence ledger ships as sources.csv in the reproducibility package.

CLOSING THOUGHT

Benchmark-led engineering references for AI systems.

EMBLEM-NLP | Reverse Engineering — Embodied Agencies

emblem.nlp | hello@emblem.nlp | @emblem_nlp